Computer Science

09. 입출력 시스템

9주차: I/O, Storage

목차 INDEX

01 I/O System

- Device
- Driver
- Polling
- Interrupt
- DMA
- Buffer
- Blocking

02 Storage

- RAM / Storage
- HDD
- SSD

03 File System

- File / Directory
- Path
- Metadata
- Inode

01
입출력 시스템
I/O System

I/O와 Device

I/O = Input / Output

입력: Input	출력: Output
키보드 · 마우스	모니터 · 스피커
파일 읽기 · 네트워크 수신	파일 쓰기 · 네트워크 송신

저장장치와 네트워크 장치는 입력과 출력 모두에 사용

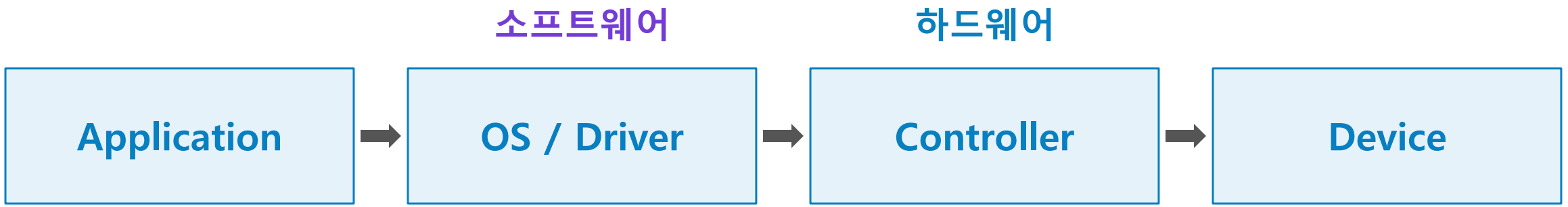
Device Controller

하드웨어



장치의 명령·상태·데이터 전송을 관리하는 제어 장치

Device Driver



Driver가 OS의 요청을 장치별 명령으로 바꾸고
Controller가 실제 장치를 제어

Polling & Interrupt



Polling

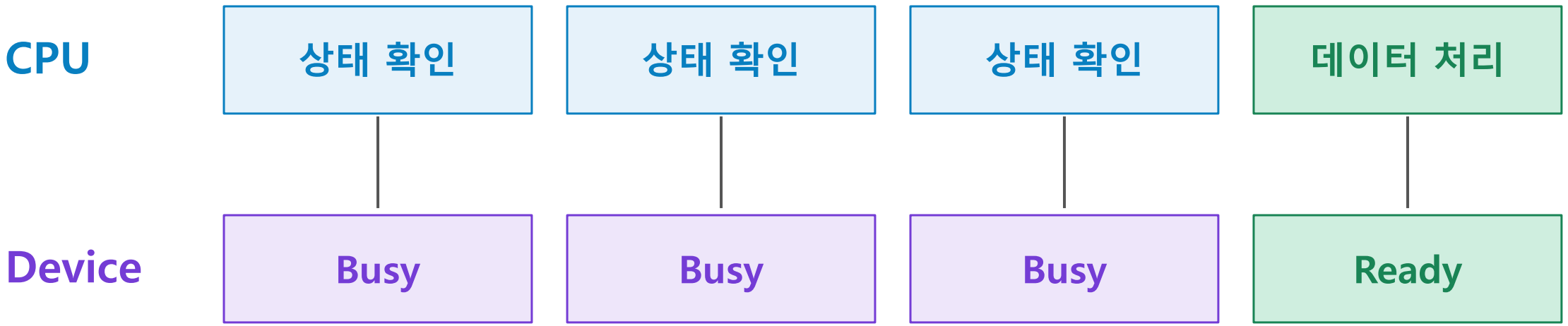
전화가 왔는지 확인하려고
일정 시간마다 전화 확인



Interrupt

전화벨이 울릴 때까지 기다림
(그동안 다른 일을 할 수 있음)

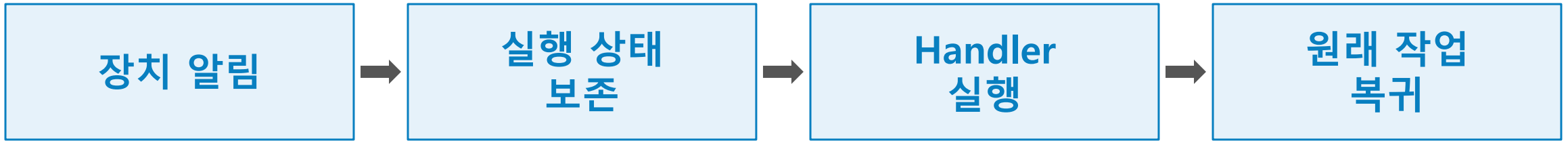
Polling



CPU가 장치 상태를 반복 확인하고, 준비되면 처리

Interrupt

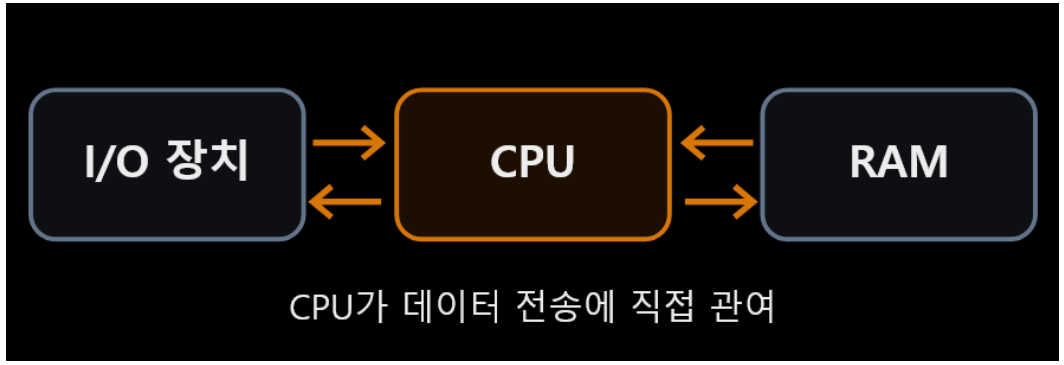
CPU는 알림을 받기 전까지 다른 작업을 수행

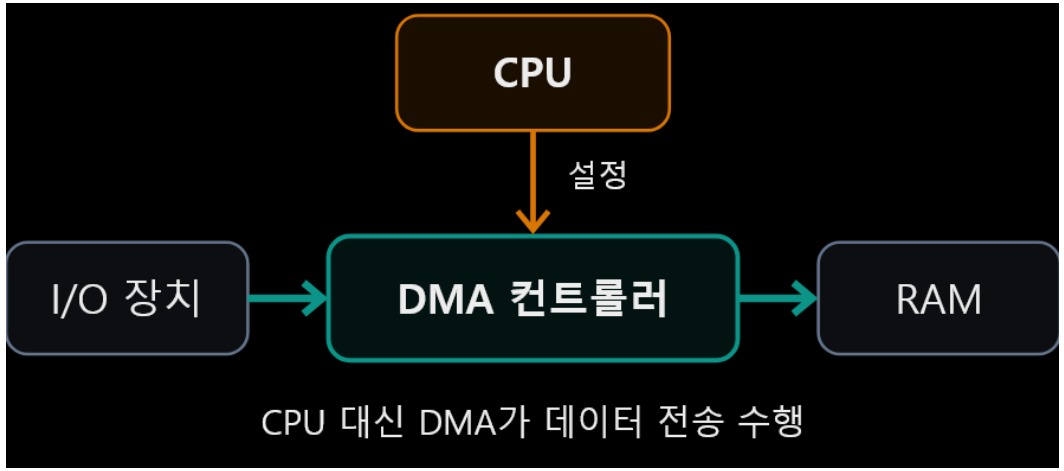


Handler 처리를 마치면 보존한 실행 상태로 돌아감

DMA: Direct Memory Access

- 입출력장치와 램 사이의 데이터 이동은 CPU를 거쳐야 함
- 문제는 CPU는 다른 일을 처리하기에도 너무 바쁨
- 따라서 입출력 장치와 램 사이를 (거의) 다이렉트로 이으려는 아이디어가 나왔고, 이것이 DMA임
- CPU를 아예 거치치 않는 것은 아니고, CPU의 코어는 거치지 않지만 메모리 컨트롤러는 거쳐감





Buffer

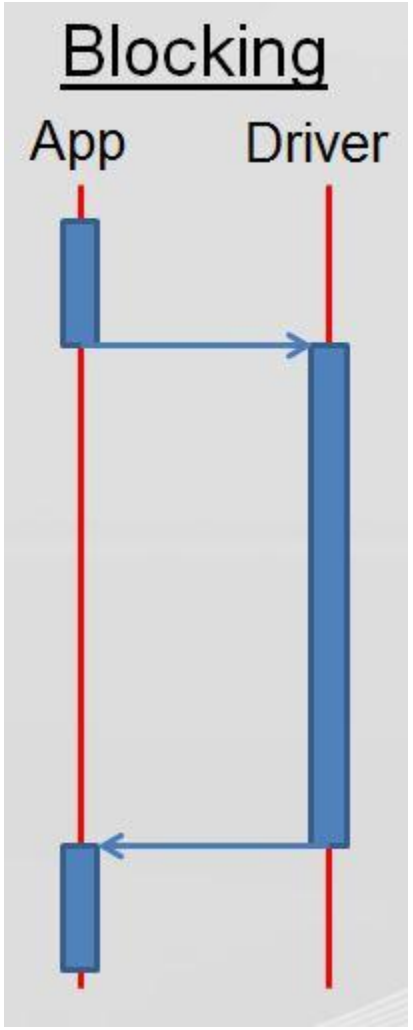


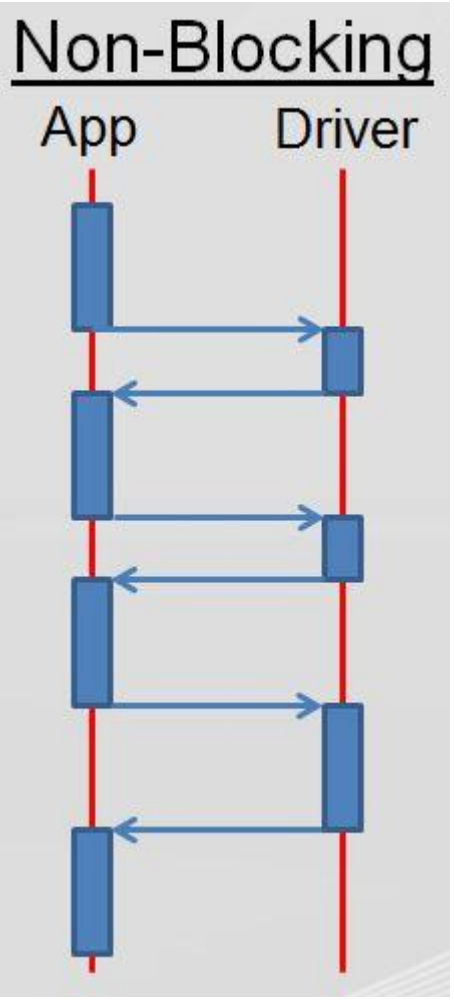
Buffer:
서로 다른 처리 속도를 연결하는 임시 저장 공간

Buffer: 전송·처리 속도 조절
Cache: 다시 사용할 데이터를 보관

Blocking과 Non-blocking I/O

- 입출력 작업을 요청했을 때, 데이터가 준비될 때까지 프로그램이 기다려야 하는가? 라는 질문에는 두 가지 해결법이 존재함
- Blocking: 해당 작업을 실행하는 스레드는 데이터가 준비될때까지 대기 [ex) input()]
- Non-Blocking: 만약 데이터가 준비되지 않았으면 기다리지 않고 일단 다른 작업을 먼저 수행함





02
저장장치
Storage

RAM과 Storage



구분	RAM	Storage
전원 차단	일반적으로 데이터 소실	데이터 유지
용도	현재 실행할 코드·데이터	프로그램·파일 보관
접근 속도	상대적으로 빠름	RAM보다 느림

RAM과 Storage



CPU

- 중앙처리장치
- 속도 매우 빠름



RAM

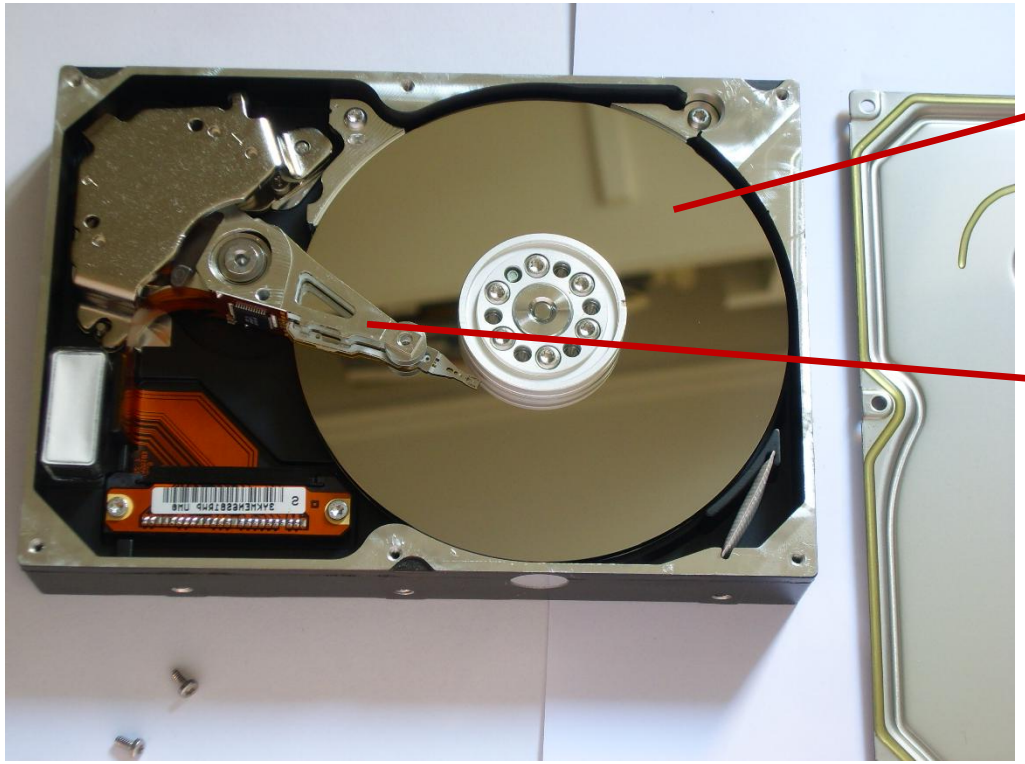
- 휘발성 메모리
(전원 차단시 소실)
- 현재 실행중인 데이터 저장
- 상대적으로 빠름
- 상대적으로 작은 용량



Storage

- 비휘발성 메모리
(전원 차단시 소실되지 않음)
- 프로그램과 파일 보관
- 상대적으로 느림
- 상대적으로 큰 용량

HDD



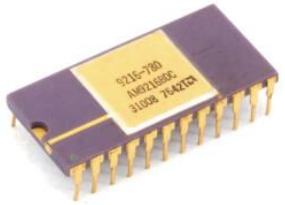
Platter

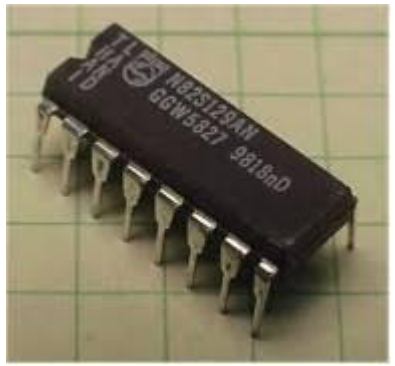
자기 방식으로 데이터를 기록하는 디스크

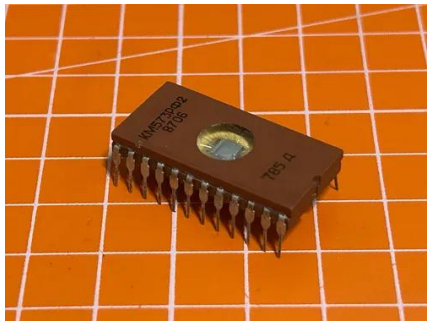
Head

원하는 트랙으로 이동한 뒤
해당 위치가 회전해 올 때까지 대기

저장장치의 발전





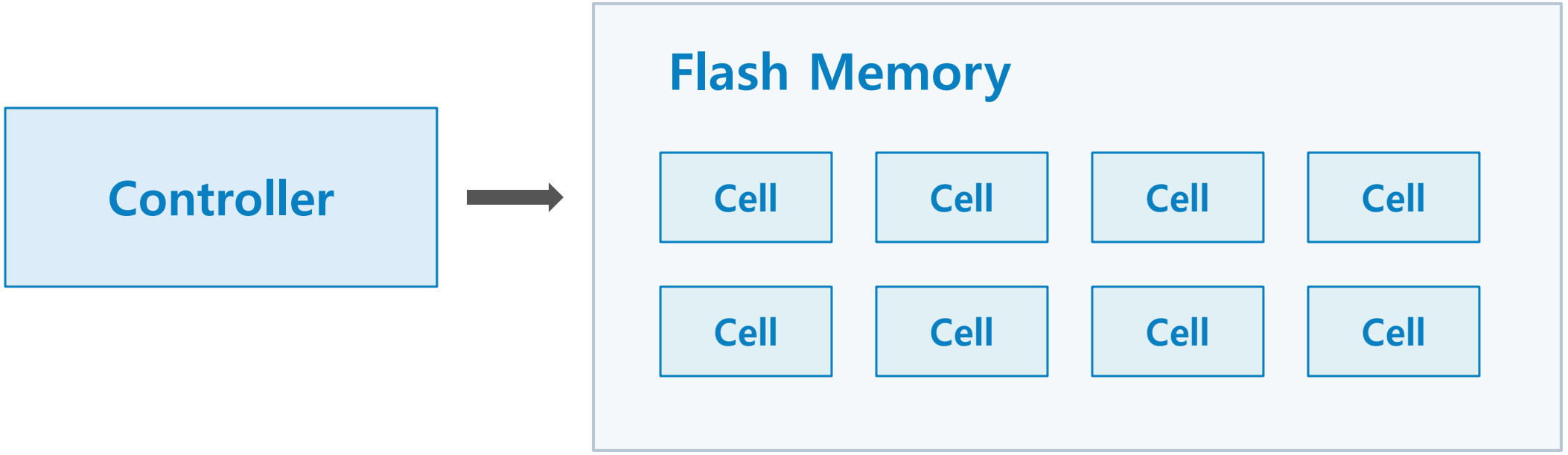






이름	Mask ROM	PROM	EPROM	EEPROM	NAND Flash
풀네임	Mask Read-Only Memory	Programmable ROM	Erasable Programmable ROM	Electrically Erasable Programmable ROM	-
기록	제조 시 기록	사용자가 1회만 기록 가능	여러 번	여러 번	여러 번
삭제	불가능	불가능	자외선으로 삭제	전기적으로 삭제	전기적으로 삭제
시기	60~70년대	70~80년대	70~90년대	80~현재	90~현재

SSD

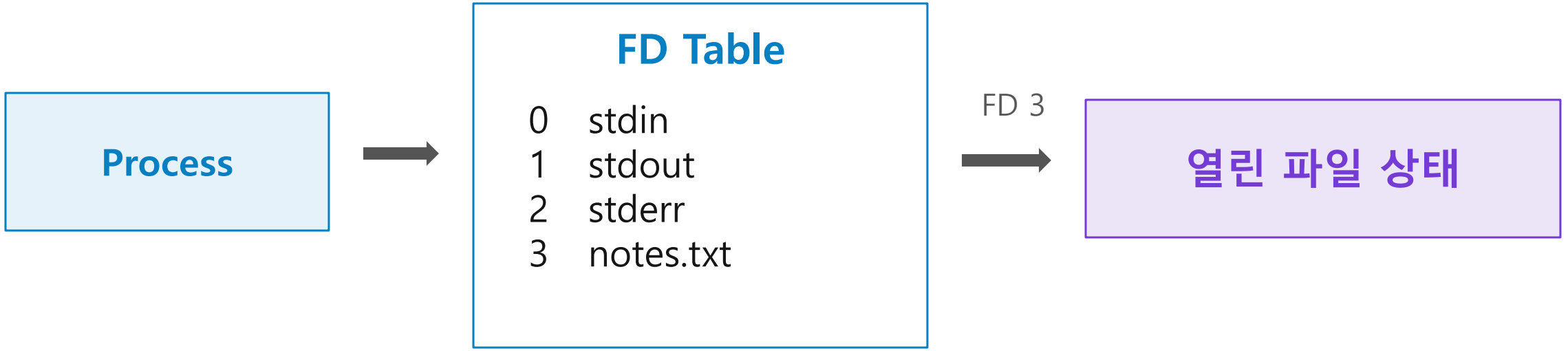


Controller가 주소 매핑과 기록, 삭제 관리
(NAND Flash)

03
File System

파일 시스템

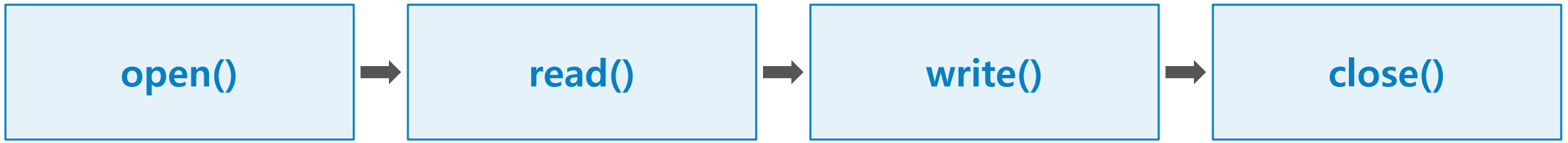
열린 파일과 File Descriptor



프로그램이 파일을 읽을 때마다 그 파일에 빠르게
다시 접근할 수 있게 번호(File Descriptor)를 발급함

그 번호들을 모아놓은 것을 FD Table이라 한다

파일 입출력 과정



경로·권한 확인
파일을 열린 상태로
전환하고 FD 생성

FD가 가리키는
파일에서
데이터 읽기

RAM의 Page Cache에
수정사항 반영
이후 저장장치에 반영

해당 FD를 닫고
파일과 관련된
자원 정리

자주 사용하는 파일 데이터는 RAM의 Page Cache에
보관되기 때문에 read()는 항상 SSD에 접근하지 않는다

Absolute Path와 Relative Path

현재 작업 디렉터리: /home/student

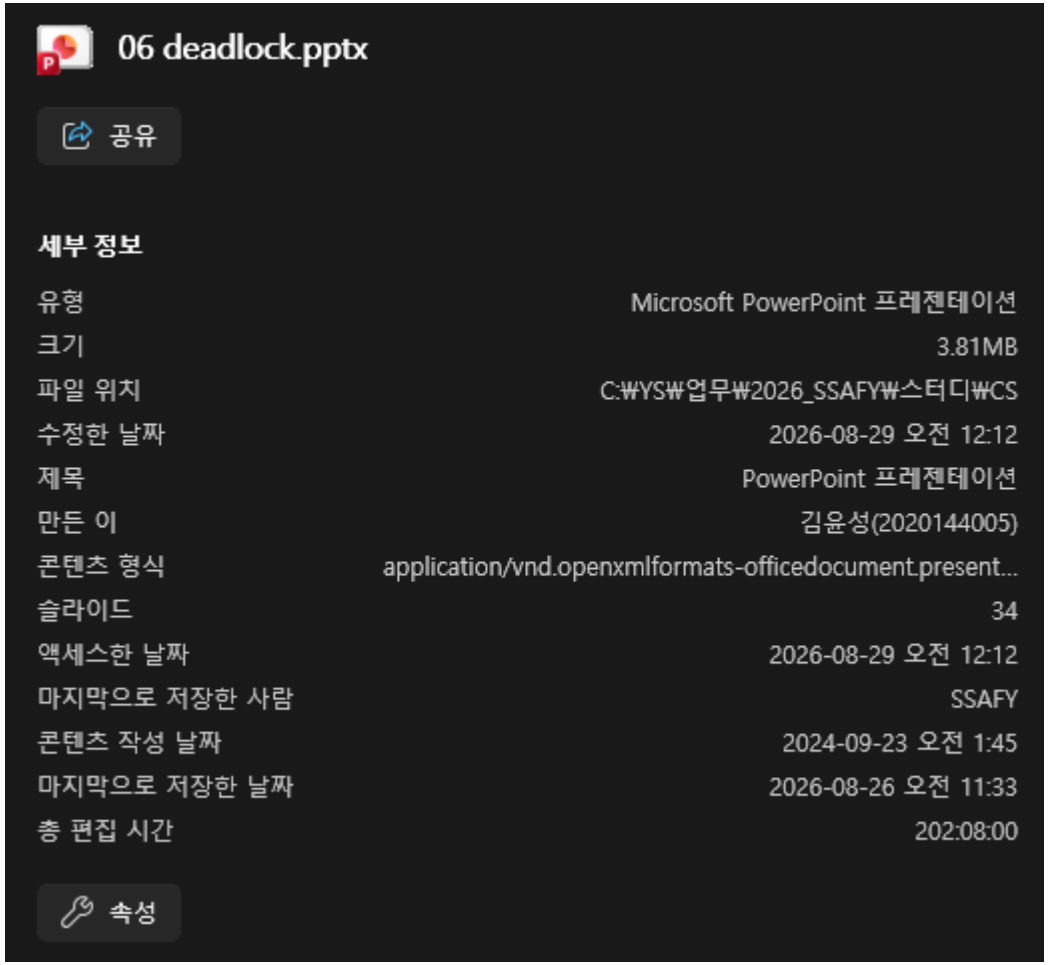
기준	같은 파일을 가리키는 경로
Absolute Path: **Root부터 지정**	/home/student/project/main.py
Relative Path: **현재 작업 위치 기준**	project/main.py

. 현재 디렉터리 .. 상위 디렉터리

File Metadata



pptx 파일



Metadata: 파일을 설명하는 부가정보

파일의 구조

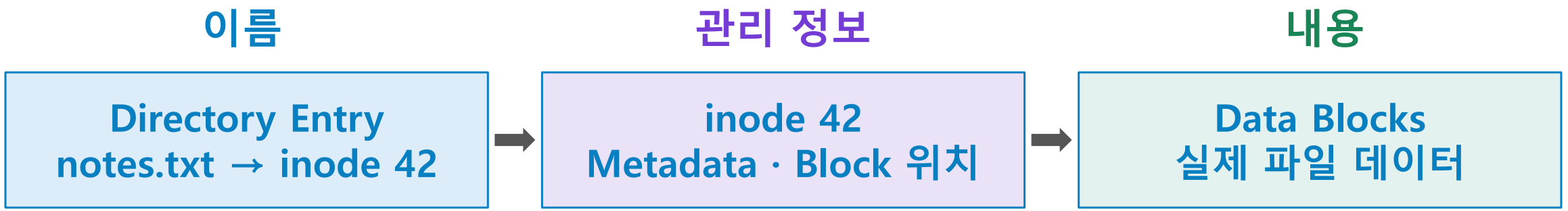


Directory Entry
파일의 이름 관리

inode (Index Node)
파일의 속성, 저장 위치 관리

Data Block
실제 파일의 내용 저장

Directory Entry와 inode



파일 이름과 inode
번호를 연결하는 정보

파일 이름으로
어떤 inode를 찾아야 하는가?

파일의 속성과 실제 데이터의
위치, 데이터를 설명하는
메타데이터 등을 저장함

보통 파일마다 inode는 1개

파일의 실제 내용 저장

파일을 읽으려면
이름 검색→ inode 조회 →
데이터 블록 접근의
절차를 거쳐야 함

파일 이름은 Directory Entry가, Metadata와 Block 위치는 inode가 관리